

Rapport de Conception : Protocole de Vote "Discovote" (Inspiré de Belenios)

- Clément Riom, Paul Lacanette, Romain Darras, Matteo Flebus

1. Introduction

Ce rapport présente la conception et la réalisation d'un nouveau protocole de vote électronique, fortement inspiré du système Belenios, mais adapté à une architecture distribuée via des bots sur la plateforme de communication Discord.

Motivation et Positionnement : La motivation principale de notre protocole est d'offrir une solution qui correspond à l'identité de notre groupe (utilisateurs quotidiens de Discord) en combinant une grande facilité d'utilisation avec un niveau de sécurité et de professionnalisme cryptographique rigoureux. Le choix de reposer sur Discord comme "Bulletin Board" et canal de communication résout d'importants défis d'infrastructure inhérents aux systèmes de vote web classiques. En déléguant la couche de transport à Discord, nous nous affranchissons des problématiques complexes d'hébergement de serveurs web, de configuration DNS, de noms de domaine, et de gestion des Autorités de Certification (certificats SSL/TLS). Le protocole est ainsi conçu pour être déployé rapidement au sein de communautés existantes pour des votes sécurisés dans la vie de tous les jours.

2. Spécification détaillée du protocole

Le système repose sur la séparation stricte des pouvoirs pour garantir l'intégrité et le secret du scrutin.

2.1 Entités participantes

- **Le Serveur d'Élection (Bot Discord) :** Gère la base de données (Urne, liste électorale), vérifie les preuves mathématiques (ZKP) et publie les résultats. Il ne possède aucune clé privée de déchiffrement des votes.
- **Les Autorités de Déchiffrement (3 Bots Discord) :** Détiennent chacune un fragment de la clé privée de l'élection. Elles doivent se coordonner pour permettre le dépouillement.
- **L'Autorité d'Enregistrement / Credential Bot :** Authentifie les votants et leur distribue de manière sécurisée (via Message Privé) un jeton de participation unique.

- **Le Votant (Application Cliente Locale)** : Interface graphique (UI) s'exécutant sur la machine physique de l'utilisateur pour garantir un chiffrement de bout en bout réel.

2.2 Primitives Cryptographiques

- **Chiffrement du vote** : ElGamal Exponentiel, choisi pour sa propriété d'homomorphisme additif permettant de compiler les résultats sans déchiffrer les bulletins individuels.
- **Preuves à Divulgaration Nulle de Connaissance (ZKP)** :
 - *Preuve disjonctive (CDS)* : Prouve que chaque case du bulletin contient exactement 0 ou 1.
 - *Preuve de somme unitaire* : Prouve que la somme des choix vaut exactement 1.
 - *Preuve de Chaum-Pedersen* : Utilisée par les autorités pour prouver qu'elles ont déchiffré l'urne honnêtement.
- **Chiffrement Hybride de bout en bout** : Les communications entre le Serveur et l'Application Cliente sont chiffrées localement via un mécanisme hybride (AES-256 pour la charge utile, RSA-2048 pour l'échange de la clé symétrique).

2.3 Déroulement (Diagramme d'échange)

1. **Initialisation** : Le Serveur crée l'élection. Les 3 Autorités publient leur clé publique partielle (h_i). Le Serveur calcule la clé publique maître (H).
2. **Enregistrement** : Le Credential Bot génère un Token (contenant H et un identifiant secret) et l'envoie chiffré en RSA en DM au votant.
3. **Vote** : Le votant colle son Token dans l'App Cliente. L'application génère le vecteur chiffré, les preuves ZKP, et crée un fichier `ballot.json` chiffré avec la clé RSA du Serveur.
4. **Dépôt** : Le votant téléverse le `ballot.json` sur le canal Discord via une commande `!server process_vote`. Le Serveur déchiffre l'enveloppe RSA, vérifie les ZKP, et insère dans l'urne.
5. **Dépouillement** : Fermeture de l'urne. Les Autorités calculent leurs parts de déchiffrement partiel (D_i). Le Serveur les multiplie et résout le logarithme discret pour afficher les résultats.

3. Description du Prototype

Pour la réalisation technique, le langage Python a été utilisé, avec les bibliothèques `discord.py` pour l'interface réseau, `cryptography` pour les primitives RSA/AES, et `tkinter` pour l'interface utilisateur cliente.

L'architecture est modulaire. Les bots s'exécutent en tant que processus indépendants et communiquent via une base de données SQLite partagée (`belenios.db`). La démonstration reproductible permet d'exécuter l'élection de bout en bout, de la génération de l'urne par l'administrateur jusqu'à la soumission du fichier JSON par le client et la certification des résultats.

4. Analyse des Propriétés de Sécurité

Conformément aux exigences, voici l'analyse rigoureuse des propriétés cryptographiques du protocole.

4.1 Propriétés Satisfaites

- **Éligibilité** : *Satisfait*. Seuls les utilisateurs possédant un `cred_public` valide dans la base de données peuvent soumettre un vote. Le double vote est impossible grâce à la colonne booléenne `has_voted`. Pour pouvoir s'inscrire au vote, il faut donc être membre du serveur discord qui l'héberge.
- **Authentification** : *Satisfait*. L'identité est vérifiée implicitement par la possession du jeton secret, distribué via les messages privés Discord (nécessitant d'être authentifié par un compte).
- **Équité (Fairness)** : *Satisfait*. Le chiffrement ElGamal empêche la lecture des bulletins. Le déchiffrement nécessite la coopération des 3 Autorités, qui n'intervient qu'après la clôture du scrutin.
- **Intégrité** : *Satisfait*. Modèle de menace : un pirate interceptant le réseau. Grâce aux preuves ZKP (CDS et Somme), tout bulletin falsifié ou modifié est mathématiquement rejeté par le Serveur lors du `process_vote`.
- **Vérifiabilité individuelle** : *Satisfait*. Le votant reçoit un hash de suivi (Tracking Number) localement sur son App. La commande `!server public_urn` permet de vérifier que ce hash est bien présent dans l'urne publique.
- **Secret du vote** : *Satisfait*. Chiffrement de bout en bout effectif. Ni Discord, ni le Serveur ne voient le choix en clair grâce à la couche RSA. L'identité Discord du votant est détachée du bulletin une fois inséré dans l'urne.

4.2 Propriétés Partiellement Satisfaites

- **Robustesse** : *Partiellement satisfait*. Notre implémentation utilise un seuil strict de 3 parmi 3 autorités. Si l'une d'elles perd sa clé ou tombe en panne, l'élection est bloquée de manière irréversible. Un véritable système de partage de secret de Shamir (type t parmi n) serait nécessaire pour une pleine robustesse.

- **Vérifiabilité universelle & d'éligibilité** : *Partiellement satisfait*. Les preuves ZKP sont incluses dans le JSON, permettant techniquement à n'importe quel auditeur de refaire les calculs. Cependant, nous supposons le Serveur honnête concernant la gestion de la table SQLite voters (Bourrage d'urne théoriquement possible par l'administrateur de la BDD).

4.3 Propriétés Non Satisfaites

- **Receipt-freeness & Résistance à la coercition** : *Non satisfait*. Le protocole ne protège pas contre la vente de vote ou la coercition physique. L'aléa cryptographique r est généré sur la machine du client, qui pourrait le conserver pour prouver son vote à un acheteur.

5. Tableau Comparatif des Propriétés

Propriété	Statut	Justification principale
Éligibilité	Satisfait	Vérification stricte en BDD (has_voted).
Authentification	Satisfait	Jeton transmis par canal sécurisé (Discord DM).
Équité	Satisfait	Chiffrement asymétrique distribué (ElGamal).
Intégrité	Satisfait	Rejet mathématique des vecteurs illégaux (ZKP).
Vérifiabilité indiv.	Satisfait	Affichage public des Tracking Numbers.
Secret du vote	Satisfait	Cryptographie côté client + Chiffrement hybride.
Robustesse	Partiellement Satisfait	Risque de blocage si une autorité fait défaut (3/3).
Vérifiabilité universelle	Partiellement Satisfait	Preuves ZKP, mais suppose le Serveur honnête.
Vérifiabilité	Partiellement Satisfait	Suppose le Serveur honnête.
Receipt-freeness	Non Satisfait	Le votant peut communiquer son tracking number pour prouver son vote.
Résistance coercition	Non Satisfait	L'aléa r est connu du client.

6. Conclusion et Limites

Le prototype démontre la viabilité d'un système électoral cryptographiquement encapsulé dans une plateforme sociale moderne. Le déport des calculs ZKP sur une interface cliente (Tkinter) permet de simuler un niveau de sécurité comparable aux protocoles académiques tout en conservant une expérience utilisateur fluide.

Limites majeures de l'implémentation : La principale limite de notre système réside dans la gestion des clés RSA pour le chiffrement hybride de la couche de transport. Actuellement, les clés RSA doivent être définies de manière statique et partagées dans un fichier de configuration partagé entre le client et le serveur *avant* le début du vote. Dans un contexte de production réel, un mécanisme dynamique d'échange de clés (comme Diffie-Hellman) ou une infrastructure à clés publiques (PKI) serait indispensable pour distribuer la clé publique du serveur à l'application cliente en temps réel, sans intervention manuelle préalable.

Pistes d'amélioration : Le remplacement du modèle d'autorité 3 parmi 3 par une signature à seuil polynomiale permettrait de régler le défaut actuel de robustesse, tolérant ainsi la défaillance d'une ou plusieurs autorités lors du dépouillement final.